Unified Blind Restoration Network (UBRN)

Single-model, single-pass restoration of jointly degraded semiconductor inspection images (speckle + Gaussian noise + downsampling, applied in randomized order).

1. Problem Formulation

Given a degraded grayscale image **Y = D(X)** where **D** is a randomized black-box composition of three degradations, learn the inverse mapping **f: Y → X■** such that X■ matches the clean full-resolution ground truth. The problem is ill-posed (one Y maps to many plausible X), the composition order is randomized (per KLA: “do not read into the order”), and the test set contains out-of-distribution samples. Scoring is on three axes: restoration quality (SSIM, PSNR, LPIPS, L1/L2), end-to-end throughput on an NVIDIA H100 (including disk I/O), and training hygiene / reproducibility.

2. Degradation Model (what the network must invert)

Degradation	Nature	Design consequence
Speckle noise	Multiplicative; scales with brightness; pushes pixels beyond [0,1]	Do NOT clip inputs (overshoot is signal). No log transform (see Flag F3).
Gaussian noise	Additive, zero-mean, variable sigma (e.g. 0.05)	Blind handling via noise-level-map conditioning (FFDNet-style).
Downsampling	512→256 or 256→128 (x2 per step)	Mandatory upsampling head; scale inferred from input shape at runtime.

3. Design Principles

- **One network, one pass.** No routing/classifier branches: DnCNN proved a single model handles blind denoising + SISR + deblocking jointly; FFDNet proved a noise-level map replaces per-noise networks. One model load also maximizes H100 throughput.
- **End-to-end on intensity domain.** No homomorphic log transform: under mixed additive+multiplicative noise, pixels can be ≤0 and log() produces NaN — an instantly invalid output. ID-CNN and Dalsasso et al. both motivate direct end-to-end learning.
- **Train on the true joint degradation.** W2S shows denoise-then-SR cascades are inefficient and SR networks are highly noise-sensitive; the joint task must be trained jointly, with the degradation pipeline randomized exactly like KLA's black box.
- **Golden-ratio sizing.** ~5–15M parameters: largest model with measurable gains, distilled/pruned if throughput demands it.
- **Fidelity first, perception second.** Pixel/SSIM/frequency losses drive training; adversarial fine-tuning is an optional, ablated stage (W2S two-phase protocol) capped to avoid hallucinated structures that KLA explicitly warns about.

4. End-to-End Pipeline

Figure 1 — UBRN inference pipeline. Every stage is justified by one of the eight referenced papers.

5. Module Specifications

Module	Specification	Source paper & adaptation
Noise estimator (optional)	3x conv3x3 (16ch, ReLU) → per-pixel noise map M; concatenated with input as channel 2. Can be dropped for fully-blind training (ablation A1).	FFDNet: tunable noise map as input to a single network — adopted as learned conditioning instead of manual routing.
Shallow extraction	Single 3x3 conv, 48 channels.	SwinIR 3-module template (shallow / deep / reconstruction).
Deep body	4-level U-shaped encoder–decoder; [4,6,6,8] blocks of MDTA (multi-Dconv-head transposed attention, CxC channel attention, linear in HxW) + GDFN (gated depthwise-conv FFN); pixel-unshuffle down / pixel-shuffle up between levels; encoder–decoder skip concats.	Restormer: MDTA + GDFN blocks and U-shaped layout, scaled down to the 5–15M param zone.
Global skip	Shallow features added to decoder output before the head; network learns the residual detail.	SwinIR Eq.5 long skip; DnCNN residual-learning principle.
Reconstruction head	conv → PixelShuffle x2 (applied once or twice for x4) → conv3x3 → 1-channel output. Scale chosen at runtime from target/input shape ratio; both heads share the body.	SwinIR sub-pixel reconstruction; W2S/ESRGAN joint denoise+SR precedent.
Output guard	torch.no_grad + eval(); nan_to_num → clip[0,1] → (H,W) float32 → same filename .npy.	KLA submission contract (valid range, no NaN/Inf, exact naming).

6. Loss Function

Primary (fidelity phase — used for the submitted model):

```
L = L_Char + 0.2 * L_SSIM + 0.1 * L_LPIPS + 0.05 * L_FFT
```

- **Charbonnier L1** ($\sqrt{(\Delta^2+\varepsilon^2)}$, ε=1e-3) — stable pixel fidelity (SwinIR Eq.7).
- **SSIM loss** — directly optimizes an officially scored metric.
- **LPIPS loss** — perceptual metric is officially scored; VGG weights bundled locally (offline rule); training-only, never loaded in run.py.
- **FFT-L1 (frequency) loss** — semiconductor patterns are periodic; KLA explicitly flagged frequency-domain signal as a promising axis.
- **Optional phase 2 (ablation A3):** relativistic-discriminator fine-tune, lambda_adv ≤ 0.005, following the W2S two-phase protocol. Submitted only if it improves LPIPS without degrading PSNR/SSIM; discriminator never ships in run.py.

7. Training Strategy

- **Randomized degradation synthesis (core OOD weapon).** Re-implement KLA's black box as the on-the-fly augmenter: random order of {Gaussian sigma in [0.01,0.1], speckle strength, blur, downsample x2/x4}, plus the 8-config flip/rotation augmentation (W2S Sec. 2.1). Never train on a fixed degradation order.
- **Mandatory sanity check first:** overfit 1–2 image pairs to perfect reconstruction; failure indicates a loader / normalization / loss bug (KLA playbook requirement).
- **Progressive learning:** start with small patches (128) & large batches, finish with large patches (256–384) & small batches (Restormer protocol) — improves large-context generalization.
- **Inputs fed raw and unclipped;** clipping applied only to final outputs.
- **Validation:** held-out split scored with PSNR + SSIM + LPIPS every epoch; visual inspection of outputs each milestone (high PSNR can coexist with hallucinated structures).
- **Reproducibility:** fixed seeds, config-file-driven runs, pip-frozen requirements.txt, optional Dockerfile (training-hygiene axis).

8. Inference & Throughput Engineering (H100, end-to-end incl. disk I/O)

- run.py <input_dir> <output_dir>: standalone .py (not a notebook); creates output dir; iterates sorted *.npy; saves identical filenames; weights loaded from local models/ folder; zero network access.
- FP16 (model.half) + torch.compile + channels_last memory format.
- Batched inference: group equal-shape .npy files into batches per forward pass.
- Background-thread prefetching of .npy files — disk I/O is inside the scored window.
- Optional: ONNX / TensorRT export if time permits (ablation A4).
- Optional self-ensemble (x8 flips/rotations) behind a flag — enabled only if the measured speed budget allows.

9. Rectified Design Flags (audit trail)

Flag	Original design	Risk	Rectification
F1	No super-resolution stage (DnCNN/FFDNet are same-resolution)	Every output at wrong resolution → automatic fail	PixelShuffle reconstruction head (SwinIR); joint training on noise+SR (W2S)
F2	Hard routing: noise classifier picks DnCNN / FFDNet / log branch	Misrouting on mixed & OOD inputs; 3 model loads hurt throughput	Single blind network + FFDNet-style noise-map conditioning
F3	Homomorphic log transform for speckle	Mixed additive noise means pixels can be ≤0 so log = NaN (invalid output); log-speckle is biased non-Gaussian (Fisher–Tippett)	In-network despeckling on intensity domain (ID-CNN, Dalsasso et al.)
F4	Conditional GAN cascade ("if pixel loss low then adversarial")	Training instability; hallucinated structures; PSNR/SSIM penalty	Fidelity-first training; optional capped adversarial fine-tune as ablation (W2S two-phase)
F5	Inputs normalized/clipped to [0,1]	Destroys speckle-overshoot signal KLA told teams to keep	Raw inputs; clip only final outputs
F6	LPIPS/VGG auto-download at first use	Evaluation machine is offline → crash	Perceptual nets are training-only; any inference weights bundled in models/

10. Ablation Plan (training-hygiene evidence)

ID	Ablation	Question answered
A1	With vs. without noise-estimator conditioning	Does explicit conditioning beat fully-blind?
A2	MDTA/GDFN body (Restormer) vs. RSTB windowed body (SwinIR)	Channel vs. windowed-spatial attention: PSNR-vs-throughput trade-off
A3	Fidelity-only vs. + adversarial fine-tune	Does GAN help LPIPS without hurting PSNR/SSIM?
A4	FP32 vs. FP16 vs. TensorRT	Throughput gain vs. quality drift
A5	Fixed vs. randomized degradation order in training	Quantifies the OOD-generalization win

11. References (the 8 uploaded papers)

- [1] Zhang et al., "Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising" (DnCNN), IEEE TIP 2017 — residual learning; single model for blind denoising + SISR + deblocking.
- [2] Zhang et al., "FFDNet: Toward a Fast and Flexible Solution for CNN-based Image Denoising", IEEE TIP 2018 — noise-level map conditioning of a single network.
- [3] Wang et al., "SAR Image Despeckling Using a Convolutional Neural Network" (ID-CNN), IEEE SPL 2017 — end-to-end despeckling on intensity domain, rejecting log-domain pipelines.
- [4] Dalsasso et al., 2020 (arXiv:2006.15559) — Fisher–Tippett analysis: log-speckle is non-Gaussian and biased.
- [5] Zhu et al., "Deep Learning Meets SAR", 2020 (arXiv:2006.10027) — learned despeckling surpasses handcrafted homomorphic filtering.
- [6] Zamir et al., "Restormer: Efficient Transformer for High-Resolution Image Restoration", CVPR 2022 (arXiv:2111.09881) — MDTA + GDFN blocks, progressive learning.
- [7] Liang et al., "SwinIR: Image Restoration Using Swin Transformer", ICCVW 2021 (arXiv:2108.10257) — shallow/deep/reconstruction template, PixelShuffle head, Charbonnier loss.

• [8] Zhou et al., "W2S: Joint Denoising and Super-Resolution" (core1 paper) — joint noise+SR must be trained jointly; two-phase GAN protocol and its PSNR trade-off.